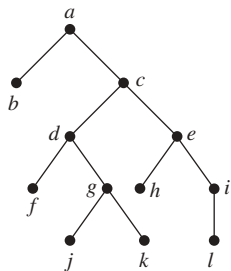


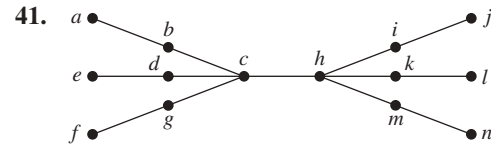
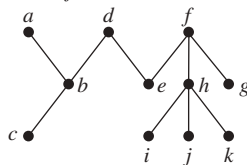
35. Answer the same questions as those given in Exercise 34 for a rooted tree representing a computer file system.
36. a) Draw the complete binary tree with 15 vertices that represents a tree-connected network of 15 processors.
b) Show how 16 numbers can be added using the 15 processors in part (a) using four steps.
37. Let n be a power of 2. Show that n numbers can be added in $\log n$ steps using a tree-connected network of $n - 1$ processors.
- *38. A **labeled tree** is a tree where each vertex is assigned a label. Two labeled trees are considered isomorphic when there is an isomorphism between them that preserves the labels of vertices. How many nonisomorphic trees are there with three vertices labeled with different integers from the set $\{1, 2, 3\}$? How many nonisomorphic trees are there with four vertices labeled with different integers from the set $\{1, 2, 3, 4\}$?

The **eccentricity** of a vertex in an unrooted tree is the length of the longest simple path beginning at this vertex. A vertex is called a **center** if no vertex in the tree has smaller eccentricity than this vertex. In Exercises 39–41 find every vertex that is a center in the given tree.

39.



40.



41. Show that a center should be chosen as the root to produce a rooted tree of minimal height from an unrooted tree.

- *43. Show that a tree has either one center or two centers that are adjacent.

44. Show that every tree can be colored using two colors.

The **rooted Fibonacci trees** T_n are defined recursively in the following way. T_1 and T_2 are both the rooted tree consisting of a single vertex, and for $n = 3, 4, \dots$, the rooted tree T_n is constructed from a root with T_{n-1} as its left subtree and T_{n-2} as its right subtree.

45. Draw the first seven rooted Fibonacci trees.

- *46. How many vertices, leaves, and internal vertices does the rooted Fibonacci tree T_n have, where n is a positive integer? What is its height?

47. What is wrong with the following “proof” using mathematical induction of the statement that every tree with n vertices has a path of length $n - 1$. *Basis step:* Every tree with one vertex clearly has a path of length 0. *Inductive step:* Assume that a tree with n vertices has a path of length $n - 1$, which has u as its terminal vertex. Add a vertex v and the edge from u to v . The resulting tree has $n + 1$ vertices and has a path of length n . This completes the inductive step.

- *48. Show that the average depth of a leaf in a binary tree with n vertices is $\Omega(\log n)$.

11.2 Applications of Trees

11.2.1 Introduction

We will discuss three problems that can be studied using trees. The first problem is: How should items in a list be stored so that an item can be easily located? The second problem is: What series of decisions should be made to find an object with a certain property in a collection of objects of a certain type? The third problem is: How should a set of characters be efficiently coded by bit strings?

11.2.2 Binary Search Trees



Searching for items in a list is one of the most important tasks that arises in computer science. Our primary goal is to implement a searching algorithm that finds items efficiently when the items are totally ordered. This can be accomplished through the use of a **binary search tree**,

which is a binary tree in which each child of a vertex is designated as a right or left child, no vertex has more than one right child or left child, and each vertex is labeled with a key, which is one of the items. Furthermore, vertices are assigned keys so that the key of a vertex is both larger than the keys of all vertices in its left subtree and smaller than the keys of all vertices in its right subtree.

This recursive procedure is used to form the binary search tree for a list of items. Start with a tree containing just one vertex, namely, the root. The first item in the list is assigned as the key of the root. To add a new item, first compare it with the keys of vertices already in the tree, starting at the root and moving to the left if the item is less than the key of the respective vertex if this vertex has a left child, or moving to the right if the item is greater than the key of the respective vertex if this vertex has a right child. When the item is less than the respective vertex and this vertex has no left child, then a new vertex with this item as its key is inserted as a new left child. Similarly, when the item is greater than the respective vertex and this vertex has no right child, then a new vertex with this item as its key is inserted as a new right child. We illustrate this procedure with Example 1.

EXAMPLE 1 Form a binary search tree for the words *mathematics*, *physics*, *geography*, *zoology*, *meteorology*, *geology*, *psychology*, and *chemistry* (using alphabetical order).

Solution: Figure 1 displays the steps used to construct this binary search tree. The word *mathematics* is the key of the root. Because *physics* comes after *mathematics* (in alphabetical order), add a right child of the root with key *physics*. Because *geography* comes before *mathematics*, add a left child of the root with key *geography*. Next, add a right child of the vertex with key *physics*, and assign it the key *zoology*, because *zoology* comes after *mathematics* and after *physics*. Similarly, add a left child of the vertex with key *physics* and assign this new vertex the key *meteorology*. Add a right child of the vertex with key *geography* and assign this new vertex the key *geology*. Add a left child of the vertex with key *zoology* and assign it the key *psychology*. Add a left child of the vertex with key *geography* and assign it the key *chemistry*. (The reader should work through all the comparisons needed at each step.)

Once we have a binary search tree, we need a way to locate items in the binary search tree, as well as a way to add new items. Algorithm 1, an insertion algorithm, actually does both of

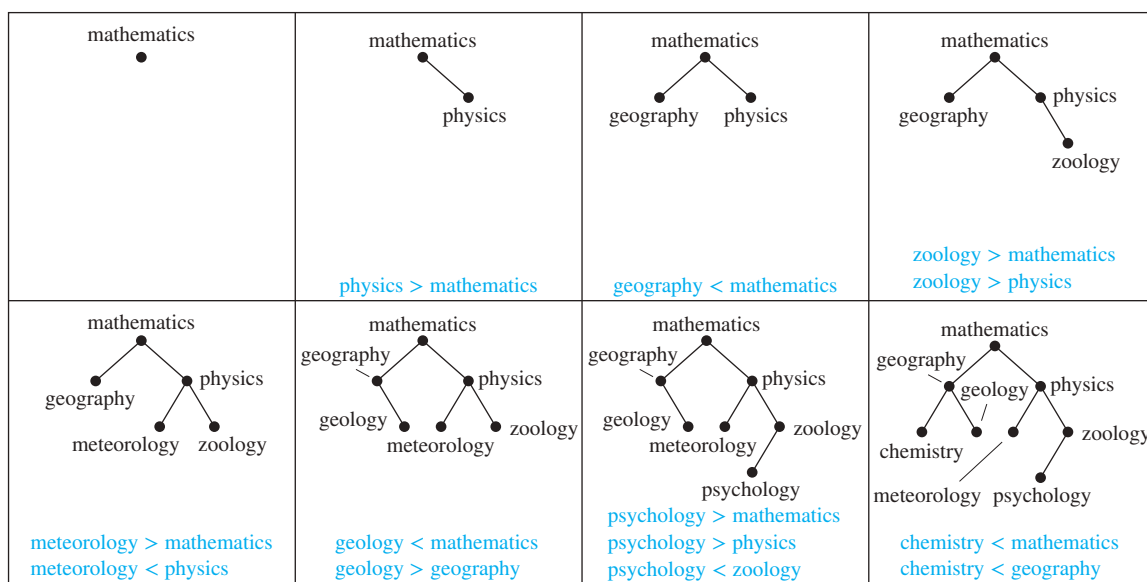


FIGURE 1 Constructing a binary search tree.

these tasks, even though it may appear that it is only designed to add vertices to a binary search tree. That is, Algorithm 1 is a procedure that locates an item x in a binary search tree if it is present, and adds a new vertex with x as its key if x is not present. In the pseudocode, v is the vertex currently under examination and $\text{label}(v)$ represents the key of this vertex. The algorithm begins by examining the root. If x equals the key of v , then the algorithm has found the location of x and terminates; if x is less than the key of v , we move to the left child of v and repeat the procedure; and if x is greater than the key of v , we move to the right child of v and repeat the procedure. If at any step we attempt to move to a child that is not present, we know that x is not present in the tree, and we add a new vertex as this child with x as its key.

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

```

procedure insertion( $T$ : binary search tree,
 $x$ : item)
 $v := \text{root of } T$ 
{a vertex not present in  $T$  has the value null }
while  $v \neq \text{null}$  and  $\text{label}(v) \neq x$ 
    if  $x < \text{label}(v)$  then
        if left child of  $v \neq \text{null}$  then  $v := \text{left child of } v$ 
        else add new vertex as a left child of  $v$  and set  $v := \text{null}$ 
    else
        if right child of  $v \neq \text{null}$  then  $v := \text{right child of } v$ 
        else add new vertex as a right child of  $v$  and set  $v := \text{null}$ 
if root of  $T = \text{null}$  then add a vertex  $v$  to the tree and label it with  $x$ 
else if  $v$  is null or  $\text{label}(v) \neq x$  then label new vertex with  $x$  and let  $v$  be this new vertex
return  $v$  { $v$  = location of  $x$ }

```

Example 2 illustrates the use of Algorithm 1 to insert a new item into a binary search tree.

EXAMPLE 2 Use Algorithm 1 to insert the word *oceanography* into the binary search tree in Example 1.

Solution: Algorithm 1 begins with v , the vertex under examination, equal to the root of T , so $\text{label}(v) = \text{mathematics}$. Because $v \neq \text{null}$ and $\text{label}(v) = \text{mathematics} < \text{oceanography}$, we next examine the right child of the root. This right child exists, so we set v , the vertex under examination, to be this right child. At this step we have $v \neq \text{null}$ and $\text{label}(v) = \text{physics} > \text{oceanography}$, so we examine the left child of v . This left child exists, so we set v , the vertex under examination, to this left child. At this step, we also have $v \neq \text{null}$ and $\text{label}(v) = \text{metereology} < \text{oceanography}$, so we try to examine the right child of v . However, this right child does not exist, so we add a new vertex as the right child of v (which at this point is the vertex with the key *metereology*) and we set $v := \text{null}$. We now exit the **while** loop because $v = \text{null}$. Because the root of T is not *null* and $v = \text{null}$, we use the **else if** statement at the end of the algorithm to label our new vertex with the key *oceanography*. ◀

We will now determine the computational complexity of this procedure. Suppose we have a binary search tree T for a list of n items. We can form a full binary tree U from T by adding unlabeled vertices whenever necessary so that every vertex with a key has two children. This is illustrated in Figure 2. Once we have done this, we can easily locate or add a new item as a key without adding a vertex.

The most comparisons needed to add a new item is the length of the longest path in U from the root to a leaf. The internal vertices of U are the vertices of T . It follows that U has n internal vertices. We can now use part (ii) of Theorem 4 in Section 11.1 to conclude that U has $n + 1$

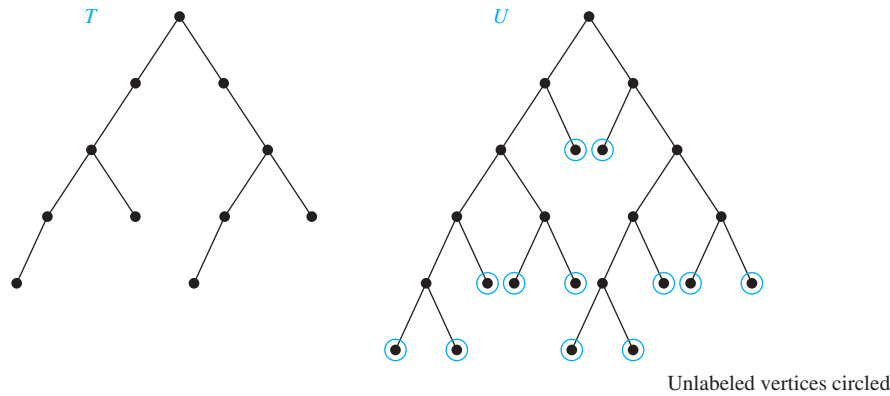


FIGURE 2 Adding unlabeled vertices to make a binary search tree full.

leaves. Using Corollary 1 of Section 11.1, we see that the height of U is greater than or equal to $h = \lceil \log_2(n + 1) \rceil$. So, there must exist an item that requires at least $\lceil \log_2(n + 1) \rceil$ comparisons to be added to the tree. Note that if U is balanced, its height is $\lceil \log_2(n + 1) \rceil$ (by Corollary 1 of Section 11.1). Thus, if a binary search tree is balanced, locating or adding an item requires no more than $\lceil \log_2(n + 1) \rceil$ comparisons. A binary search tree can become unbalanced as items are added to it. Because balanced binary search trees give optimal worst-case complexity for binary searching, algorithms have been devised that rebalance binary search trees as items are added. The interested reader can consult references on data structures for the description of such algorithms.

11.2.3 Decision Trees



Rooted trees can be used to model problems in which a series of decisions leads to a solution. For instance, a binary search tree can be used to locate items based on a series of comparisons, where each comparison tells us whether we have located the item, or whether we should go right or left in a subtree. A rooted tree in which each internal vertex corresponds to a decision, with a subtree at these vertices for each possible outcome of the decision, is called a **decision tree**. The possible solutions of the problem correspond to the paths to the leaves of this rooted tree. Example 3 illustrates an application of decision trees.

EXAMPLE 3



Suppose there are seven coins, all with the same weight, and a counterfeit coin that weighs less than the others. How many weighings are necessary using a balance scale to determine which of the eight coins is the counterfeit one? Give an algorithm for finding this counterfeit coin.

Solution: There are three possibilities for each weighing on a balance scale. The two pans can have equal weight, the first pan can be heavier, or the second pan can be heavier. Consequently, the decision tree for the sequence of weighings is a 3-ary tree. There are at least eight leaves in the decision tree because there are eight possible outcomes (because each of the eight coins can be the counterfeit lighter coin), and each possible outcome must be represented by at least one leaf. The largest number of weighings needed to determine the counterfeit coin is the height of the decision tree. From Corollary 1 of Section 11.1 it follows that the height of the decision tree is at least $\lceil \log_3 8 \rceil = 2$. Hence, at least two weighings are needed.

It is possible to determine the counterfeit coin using two weighings. The decision tree that illustrates how this is done is shown in Figure 3. ◀

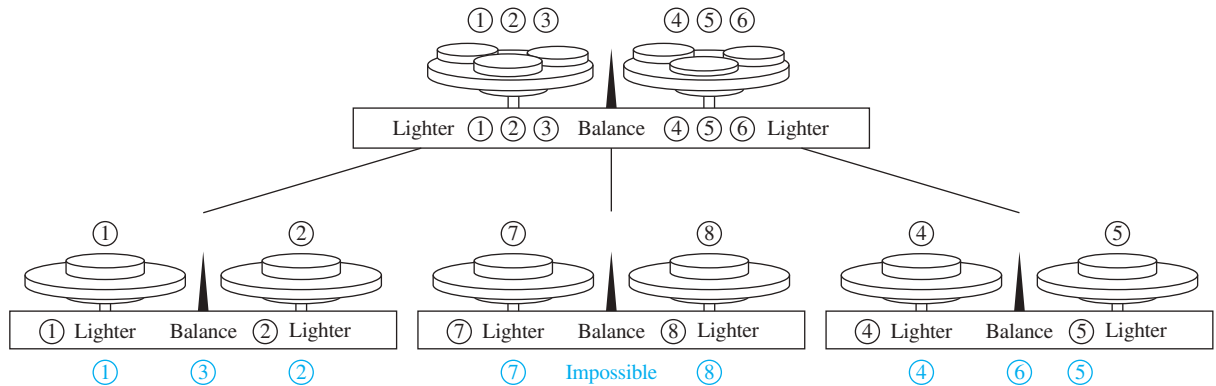


FIGURE 3 A decision tree for locating a counterfeit coin. The counterfeit coin is shown in color below each final weighing.

THE COMPLEXITY OF COMPARISON-BASED SORTING ALGORITHMS Many different sorting algorithms have been developed. To decide whether a particular sorting algorithm is efficient, its complexity is determined. Using decision trees as models, a lower bound for the worst-case complexity of sorting algorithms that are based on binary comparisons can be found.

We can use decision trees to model sorting algorithms and to determine an estimate for the worst-case complexity of these algorithms. Note that given n elements, there are $n!$ possible orderings of these elements, because each of the $n!$ permutations of these elements can be the correct order. The sorting algorithms studied in this book, and most commonly used sorting algorithms, are based on binary comparisons, that is, the comparison of two elements at a time. The result of each such comparison narrows down the set of possible orderings. Thus, a sorting algorithm based on binary comparisons can be represented by a binary decision tree in which each internal vertex represents a comparison of two elements. Each leaf represents one of the $n!$ permutations of n elements.

EXAMPLE 4 We display in Figure 4 a decision tree that orders the elements of the list a, b, c . ◀

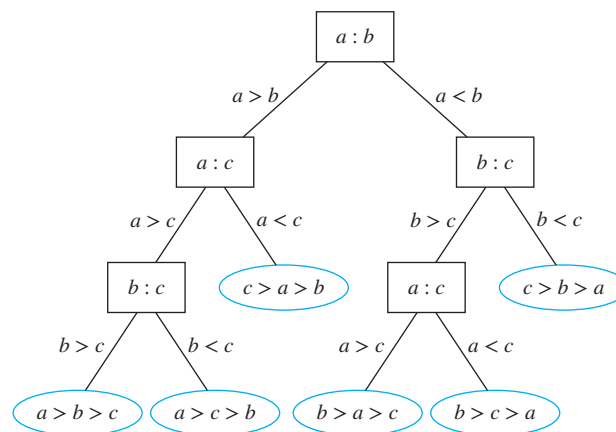


FIGURE 4 A decision tree for sorting three distinct elements.

The complexity of a sort based on binary comparisons is measured in terms of the number of such comparisons used. The largest number of binary comparisons ever needed to sort a list with n elements gives the worst-case performance of the algorithm. The most comparisons used

equals the longest path length in the decision tree representing the sorting procedure. In other words, the largest number of comparisons ever needed is equal to the height of the decision tree. Because the height of a binary tree with $n!$ leaves is at least $\lceil \log_2 n! \rceil$ (using Corollary 1 in Section 11.1), at least $\lceil \log_2 n! \rceil$ comparisons are needed, as stated in Theorem 1.

THEOREM 1

A sorting algorithm based on binary comparisons requires at least $\lceil \log_2 n! \rceil$ comparisons.

We can use Theorem 1 to provide a big-Omega estimate for the number of comparisons used by a sorting algorithm based on binary comparison. We need only note that by Exercise 74 in Section 3.2 we know that $\lceil \log n! \rceil$ is $\Theta(n \log n)$, one of the commonly used reference functions for the computational complexity of algorithms. Corollary 1 is a consequence of this estimate.

COROLLARY 1

The number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log n)$.

A consequence of Corollary 1 is that a sorting algorithm based on binary comparisons that uses $\Theta(n \log n)$ comparisons, in the worst case, to sort n elements is optimal, in the sense that no other such algorithm has better worst-case complexity. Note that by Theorem 1 in Section 5.4 we see that the merge sort algorithm is optimal in this sense.

We can also establish a similar result for the average-case complexity of sorting algorithms. The average number of comparisons used by a sorting algorithm based on binary comparisons is the average depth of a leaf in the decision tree representing the sorting algorithm. By Exercise 48 in Section 11.1 we know that the average depth of a leaf in a binary tree with N vertices is $\Omega(\log N)$. We obtain the following estimate when we let $N = n!$ and note that a function that is $\Omega(\log n!)$ is also $\Omega(n \log n)$ because $\log n!$ is $\Theta(n \log n)$.

THEOREM 2

The average number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log n)$.

11.2.4 Prefix Codes

Consider the problem of using bit strings to encode the letters of the English alphabet (where no distinction is made between lowercase and uppercase letters). We can represent each letter with a bit string of length five, because there are only 26 letters and there are 32 bit strings of length five. The total number of bits used to encode data is five times the number of characters in the text when each character is encoded with five bits. Is it possible to find a coding scheme of these letters such that, when data are coded, fewer bits are used? We can save memory and reduce transmittal time if this can be done.

Consider using bit strings of different lengths to encode letters. Letters that occur more frequently should be encoded using short bit strings, and longer bit strings should be used to encode rarely occurring letters. When letters are encoded using varying numbers of bits, some method must be used to determine where the bits for each character start and end. For instance, if e were encoded with 0, a with 1, and t with 01, then the bit string 0101 could correspond to *eat*, *tea*, *eaea*, or *tt*.

One way to ensure that no bit string corresponds to more than one sequence of letters is to encode letters so that the bit string for a letter never occurs as the first part of the bit string for another letter. Codes with this property are called **prefix codes**. For instance, the encoding of e

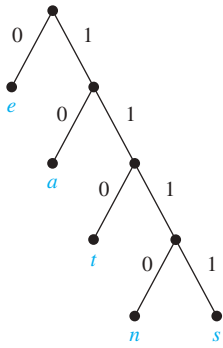


FIGURE 5 A binary tree with a prefix code.

as 0, *a* as 10, and *t* as 11 is a prefix code. A word can be recovered from the unique bit string that encodes its letters. For example, the string 10110 is the encoding of *ate*. To see this, note that the initial 1 does not represent a character, but 10 does represent *a* (and could not be the first part of the bit string of another letter). Then, the next 1 does not represent a character, but 11 does represent *t*. The final bit, 0, represents *e*.

A prefix code can be represented using a binary tree, where the characters are the labels of the leaves in the tree. The edges of the tree are labeled so that an edge leading to a left child is assigned a 0 and an edge leading to a right child is assigned a 1. The bit string used to encode a character is the sequence of labels of the edges in the unique path from the root to the leaf that has this character as its label. For instance, the tree in Figure 5 represents the encoding of *e* by 0, *a* by 10, *t* by 110, *n* by 1110, and *s* by 1111.

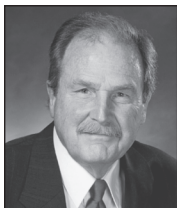
The tree representing a code can be used to decode a bit string. For instance, consider the word encoded by 11111011100 using the code in Figure 5. This bit string can be decoded by starting at the root, using the sequence of bits to form a path that stops when a leaf is reached. Each 0 bit takes the path down the edge leading to the left child of the last vertex in the path, and each 1 bit corresponds to the right child of this vertex. Consequently, the initial 1111 corresponds to the path starting at the root, going right four times, leading to a leaf in the graph that has *s* as its label, because the string 1111 is the code for *s*. Continuing with the fifth bit, we reach a leaf next after going right then left, when the vertex labeled with *a*, which is encoded by 10, is visited. Starting with the seventh bit, we reach a leaf next after going right three times and then left, when the vertex labeled with *n*, which is encoded by 1110, is visited. Finally, the last bit, 0, leads to the leaf that is labeled with *e*. Therefore, the original word is *sane*.

We can construct a prefix code from any binary tree where the left edge at each internal vertex is labeled by 0 and the right edge by a 1 and where the leaves are labeled by characters. Characters are encoded with the bit string constructed using the labels of the edges in the unique path from the root to the leaves.

Links

HUFFMAN CODING We now introduce an algorithm that takes as input the frequencies (which are the probabilities of occurrences) of symbols in a string and produces as output a prefix code that encodes the string using the fewest possible bits, among all possible binary prefix codes for these symbols. This algorithm, known as **Huffman coding**, was developed by David Huffman in a term paper he wrote in 1951 while a graduate student at MIT. (Note that this algorithm assumes that we already know how many times each symbol occurs in the string, so we can compute the frequency of each symbol by dividing the number of times this symbol occurs by the length of the string.) Huffman coding is a fundamental algorithm in *data compression*, the subject devoted to reducing the number of bits required to represent information. Huffman coding is extensively used to compress bit strings representing text and it also plays an important role in compressing audio and image files.

Links



Courtesy of California State University

DAVID A. HUFFMAN (1925–1999) David Huffman grew up in Ohio. At the age of 18 he received his B.S. in electrical engineering from The Ohio State University. Afterward he served in the U.S. Navy as a radar maintenance officer on a destroyer that had the mission of clearing mines in Asian waters after World War II. Later, he earned his M.S. from Ohio State and his Ph.D. in electrical engineering from MIT. Huffman joined the MIT faculty in 1953, where he remained until 1967, when he became the founding member of the computer science department at the University of California at Santa Cruz. He played an important role in developing this department and spent the remainder of his career there, retiring in 1994.

Huffman is noted for his contributions to information theory and coding, signal designs for radar and for communications, and design procedures for asynchronous logical circuits. His work on surfaces with zero curvature led him to develop original techniques for folding paper and vinyl into unusual shapes considered works of art by many and publicly displayed in several exhibits. However, Huffman is best known for his development of what is now called Huffman coding, a result of a term paper he wrote during his graduate work at MIT.

Huffman enjoyed exploring the outdoors, hiking, and traveling extensively. He became certified as a scuba diver when he was in his late 60s. He kept poisonous snakes as pets.

Algorithm 2 presents the Huffman coding algorithm. Given symbols and their frequencies, our goal is to construct a rooted binary tree where the symbols are the labels of the leaves. The algorithm begins with a forest of trees each consisting of one vertex, where each vertex has a symbol as its label and where the weight of this vertex equals the frequency of the symbol that is its label. At each step, we combine two trees having the least total weight into a single tree by introducing a new root and placing the tree with larger weight as its left subtree and the tree with smaller weight as its right subtree. Furthermore, we assign the sum of the weights of the two subtrees of this tree as the total weight of the tree. (Although procedures for breaking ties by choosing between trees with equal weights can be specified, we will not specify such procedures here.) The algorithm is finished when it has constructed a tree, that is, when the forest is reduced to a single tree.

Demo 

ALGORITHM 2 Huffman Coding.

procedure *Huffman*(C : symbols a_i with frequencies $w_i, i = 1, \dots, n$)
 $F :=$ forest of n rooted trees, each consisting of the single vertex a_i and assigned weight w_i
while F is not a tree
 Replace the rooted trees T and T' of least weights from F with $w(T) \geq w(T')$ with a tree having a new root that has T as its left subtree and T' as its right subtree. Label the new edge to T with 0 and the new edge to T' with 1.
 Assign $w(T) + w(T')$ as the weight of the new tree.
{the Huffman coding for the symbol a_i is the concatenation of the labels of the edges in the unique path from the root to the vertex a_i }

Example 5 illustrates how Algorithm 2 is used to encode a set of five symbols.

EXAMPLE 5 Use Huffman coding to encode the following symbols with the frequencies listed: A: 0.08, B: 0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35. What is the average number of bits used to encode a character?

Extra Examples 

Solution: Figure 6 displays the steps used to encode these symbols. The encoding produced encodes A by 111, B by 110, C by 011, D by 010, E by 10, and F by 00. The average number of bits used to encode a symbol using this encoding is

$$3 \cdot 0.08 + 3 \cdot 0.10 + 3 \cdot 0.12 + 3 \cdot 0.15 + 2 \cdot 0.20 + 2 \cdot 0.35 = 2.45.$$

Note that Huffman coding is a greedy algorithm. Replacing the two subtrees with the smallest weight at each step leads to an optimal code in the sense that no binary prefix code for these symbols can encode these symbols using fewer bits. We leave the proof that Huffman codes are optimal as Exercise 32.

There are many variations of Huffman coding. For example, instead of encoding single symbols, we can encode blocks of symbols of a specified length, such as blocks of two symbols. Doing so may reduce the number of bits required to encode the string (see Exercise 30). We can also use more than two symbols to encode the original symbols in the string (see the preamble to Exercise 28). Furthermore, a variation known as adaptive Huffman coding (see [Sa00]) can be used when the frequency of each symbol in a string is not known in advance, so that encoding is done at the same time the string is being read.

Huffman coding is used in JPEG image coding.

11.2.5 Game Trees

Links 

Trees can be used to analyze certain types of games such as tic-tac-toe, nim, checkers, and chess. In each of these games, two players take turns making moves. Each player knows the

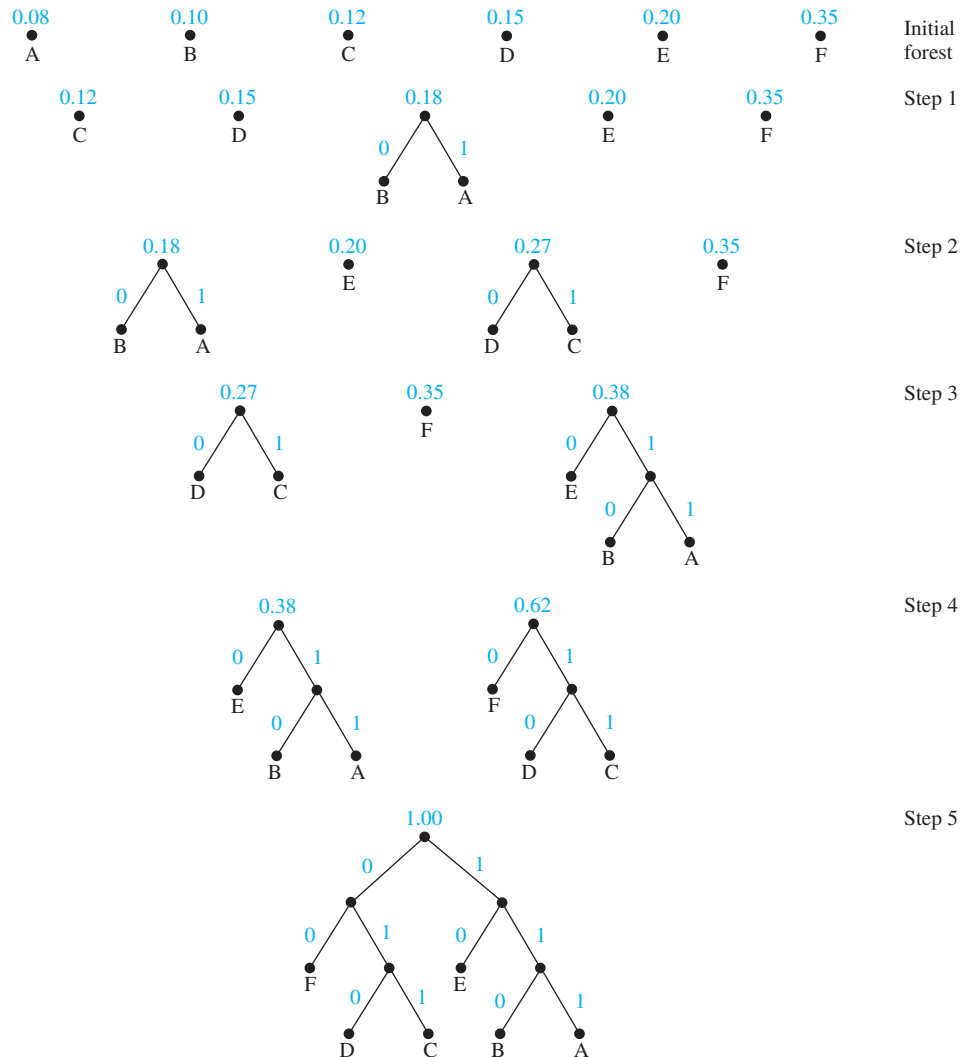


FIGURE 6 Huffman coding of symbols in Example 4.

moves made by the other player and no element of chance enters into the game. We model such games using **game trees**; the vertices of these trees represent the positions that a game can be in as it progresses; the edges represent legal moves between these positions. Because game trees are usually large, we simplify game trees by representing all symmetric positions of a game by the same vertex. However, the same position of a game may be represented by different vertices if different sequences of moves lead to this position. The root represents the starting position. The usual convention is to represent vertices at even levels by boxes and vertices at odd levels by circles. When the game is in a position represented by a vertex at an even level, it is the first player's move; when the game is in a position represented by a vertex at an odd level, it is the second player's move. Game trees may be infinite when the games they represent never end, such as games that can enter infinite loops, but for most games there are rules that lead to finite game trees.

The leaves of a game tree represent the final positions of a game. We assign a value to each leaf indicating the payoff to the first player if the game terminates in the position represented by this leaf. For games that are win-lose, we label a terminal vertex represented by a circle with a 1 to indicate a win by the first player and we label a terminal vertex represented by a box with

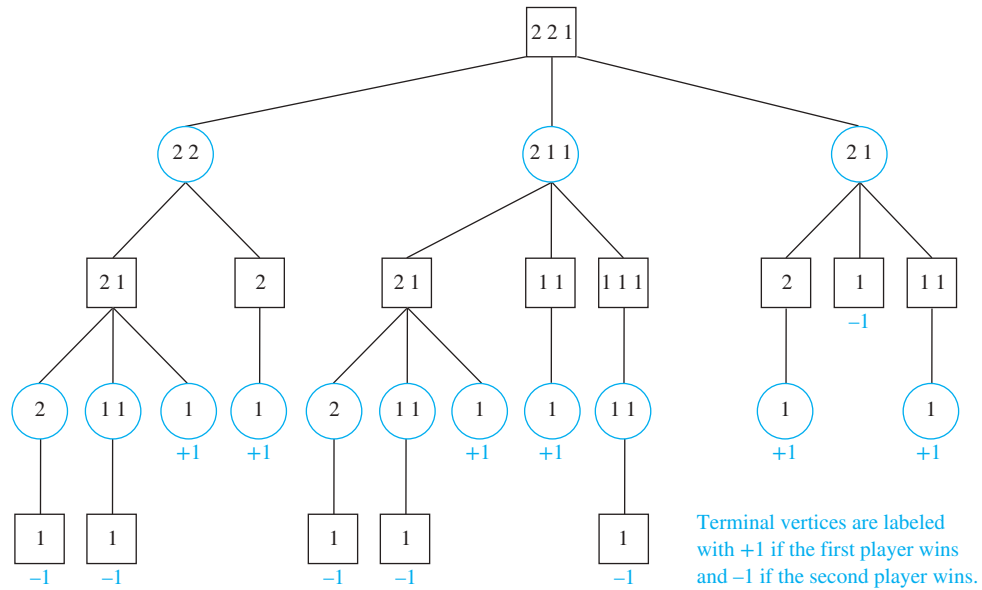


FIGURE 7 The game tree for a game of nim.

a -1 to indicate a win by the second player. For games where draws are allowed, we label a terminal vertex corresponding to a draw position with a 0 . Note that for win–lose games, we have assigned values to terminal vertices so that the larger the value, the better the outcome for the first player.

In Example 6 we display a game tree for a well-known and well-studied game.

EXAMPLE 6

Nim In a version of the game of **nim**, at the start of a game there are a number of piles of stones. Two players take turns making moves; a legal move consists of removing one or more stones from one of the piles, without removing all the stones left. A player without a legal move loses. (Another way to look at this is that the player removing the last stone loses because the position with no piles of stones is not allowed.) The game tree shown in Figure 7 represents this version of nim given the starting position where there are three piles of stones containing two, two, and one stone each, respectively. We represent each position with an unordered list of the number of stones in the different piles (the order of the piles does not matter). The initial move by the first player can lead to three possible positions because this player can remove one stone from a pile with two stones (leaving three piles containing one, one, and two stones); two stones from a pile containing two stones (leaving two piles containing two stones and one stone); or one stone from the pile containing one stone (leaving two piles of two stones). When only one pile with one stone is left, no legal moves are possible, so such positions are terminal positions. Because nim is a win–lose game, we label the terminal vertices with $+1$ when they represent wins for the first player and -1 when they represent wins for the second player. ◀

Although nim is an ancient game, Charles Bouton coined its modern name in 1901 after an archaic English word meaning “to steal.”

EXAMPLE 7

Tic-tac-toe The game tree for tic-tac-toe is extremely large and cannot be drawn here, although a computer could easily build such a tree. We show a portion of the game tree for tic-tac-toe in Figure 8(a). Note that by considering symmetric positions equivalent, we need only consider three possible initial moves, as shown in Figure 8(a). We also show a subtree of this game tree leading to terminal positions in Figure 8(b), where a player who can win makes a winning move. ◀

We can recursively define the values of all vertices in a game tree in a way that enables us to determine the outcome of this game when both players follow optimal strategies. By a

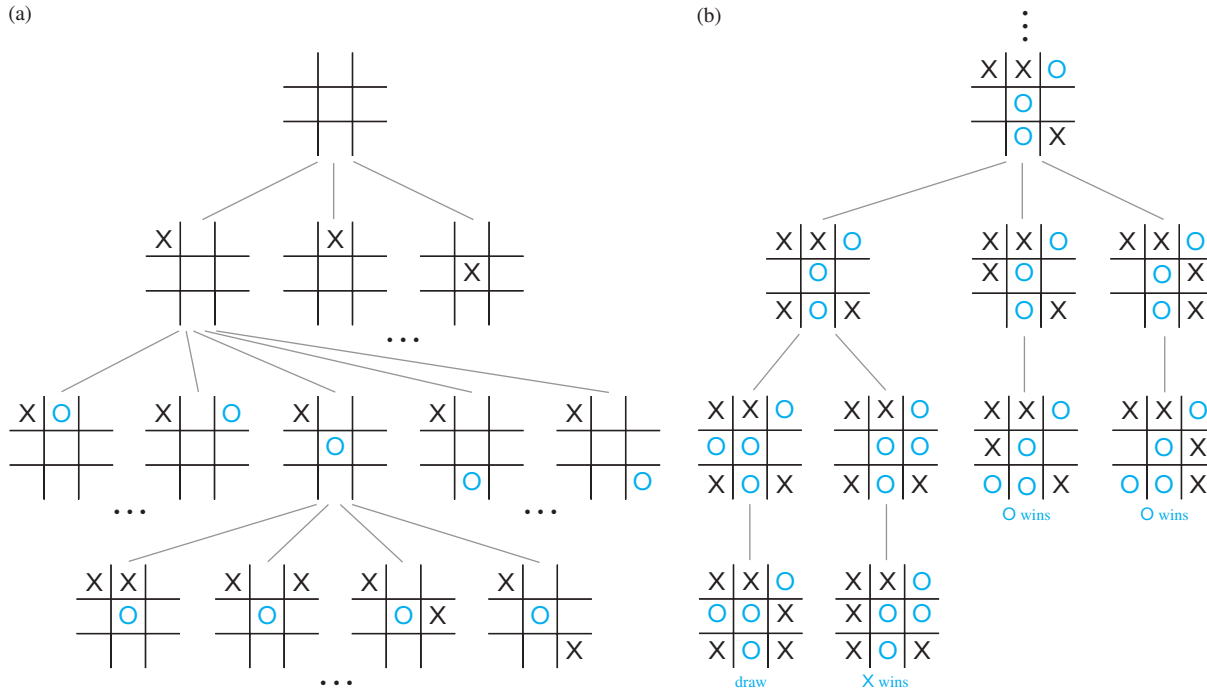


FIGURE 8 Some of the game tree for tic-tac-toe.

strategy we mean a set of rules that tells a player how to select moves to win the game. An optimal strategy for the first player is a strategy that maximizes the payoff to this player and for the second player is a strategy that minimizes this payoff. We now recursively define the value of a vertex.

Definition 1

The *value of a vertex in a game tree* is defined recursively as:

- (i) the value of a leaf is the payoff to the first player when the game terminates in the position represented by this leaf.
- (ii) the value of an internal vertex at an even level is the maximum of the values of its children, and the value of an internal vertex at an odd level is the minimum of the values of its children.

The strategy where the first player moves to a position represented by a child with maximum value and the second player moves to a position of a child with minimum value is called the **minmax strategy**. We can determine who will win the game when both players follow the minmax strategy by calculating the value of the root of the tree; this value is called the **value** of the tree. This is a consequence of Theorem 3.

THEOREM 3

The value of a vertex of a game tree tells us the payoff to the first player if both players follow the minmax strategy and play starts from the position represented by this vertex.

Proof: We will use induction to prove this theorem.

BASIS STEP: If the vertex is a leaf, by definition the value assigned to this vertex is the payoff to the first player.

INDUCTIVE STEP: The inductive hypothesis is the assumption that the values of the children of a vertex are the payoffs to the first player, assuming that play starts at each of the positions represented by these vertices. We need to consider two cases, when it is the first player's turn and when it is the second player's turn.

When it is the first player's turn, this player follows the minmax strategy and moves to the position represented by the child with the largest value. By the inductive hypothesis, this value is the payoff to the first player when play starts at the position represented by this child and follows the minmax strategy. By the recursive step in the definition of the value of an internal vertex at an even level (as the maximum value of its children), the value of this vertex is the payoff when play begins at the position represented by this vertex.

When it is the second player's turn, this player follows the minmax strategy and moves to the position represented by the child with the least value. By the inductive hypothesis, this value is the payoff to the first player when play starts at the position represented by this child and both players follow the minmax strategy. By the recursive definition of the value of an internal vertex at an odd level as the minimum value of its children, the value of this vertex is the payoff when play begins at the position represented by this vertex. ◀

Remark: By extending the proof of Theorem 3, it can be shown that the minmax strategy is the optimal strategy for both players.

Example 8 illustrates how the minmax procedure works. It displays the values assigned to the internal vertices in the game tree from Example 6. Note that we can shorten the computation required by noting that for win-lose games, once a child of a square vertex with value $+1$ is found, the value of the square vertex is also $+1$ because $+1$ is the largest possible payoff. Similarly, once a child of a circle vertex with value -1 is found, this is the value of the circle vertex also.

EXAMPLE 8 In Example 6 we constructed the game tree for nim with a starting position where there are three piles containing two, two, and one stones. In Figure 9 we show the values of the vertices of this game tree. The values of the vertices are computed using the values of the leaves and working one level up at a time. In the right margin of this figure we indicate whether we use the maximum or minimum of the values of the children to find the value of an internal vertex at

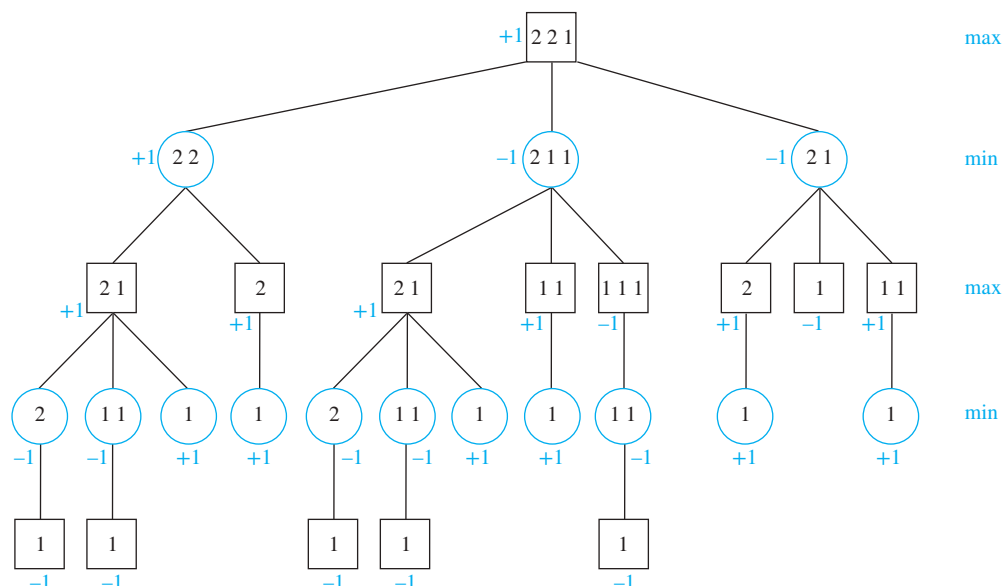


FIGURE 9 Showing the values of vertices in the game of nim.

each level. For example, once we have found the values of the three children of the root, which are 1, -1 , and -1 , we find the value of the root by computing $\max(1, -1, -1) = 1$. Because the value of the root is 1, it follows that the first player wins when both players follow a minmax strategy. 

Game trees for some well-known games can be extraordinarily large, because these games have many different possible moves. For example, the game tree for chess has been estimated to have as many as 10^{100} vertices! It may be impossible to use Theorem 3 directly to study a game because of the size of the game tree. Therefore, various approaches have been devised to help determine good strategies and to determine the outcome of such games. One useful technique, called *alpha-beta pruning*, eliminates much computation by pruning portions of the game tree that cannot affect the values of ancestor vertices. (For information about alpha-beta pruning, consult [Gr90].) Another useful approach is to use *evaluation functions*, which estimate the value of internal vertices in the game tree when it is not feasible to compute these values exactly. For example, in the game of tic-tac-toe, as an evaluation function for a position, we may use the number of files (rows, columns, and diagonals) containing no Os (used to indicate moves of the second player) minus the number of files containing no Xs (used to indicate moves of the first player). This evaluation function provides some indication of which player has the advantage in the game. Once the values of an evaluation function are inserted, the value of the game can be computed following the rules used for the minmax strategy. Computer programs created to play chess, such as IBM's Deep Blue, the first chess-playing computer to win a match against a current world champion under regular rules, are based on sophisticated evaluation functions. For more information about how computers play chess see [Le91].

Chess programs on smartphones can now play at the grandmaster level.

Links 

Links 

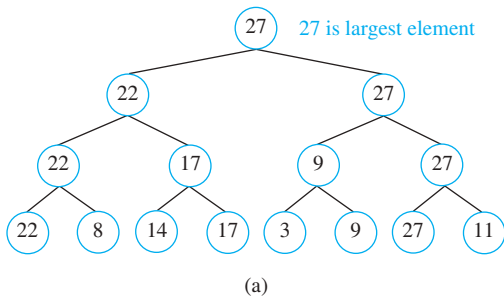
The material we have studied comes from **combinatorial game theory**, which deals with games where a player knows all previous moves and selects a move before other players choose theirs. For more information about combinatorial game theory, consult [AlNoWo07], [BeCoGu82a, 82b], or [Be04], and the web links for this subject.

Exercises

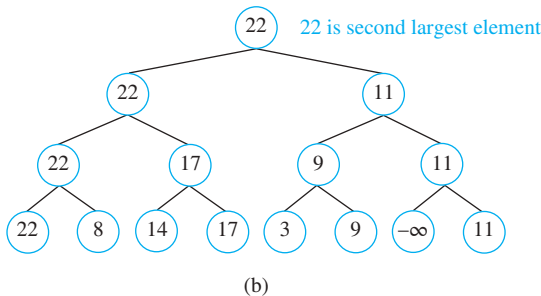
- Build a binary search tree for the words *banana*, *peach*, *apple*, *pear*, *coconut*, *mango*, and *papaya* using alphabetical order.
- Build a binary search tree for the words *oenology*, *phrenology*, *campanology*, *ornithology*, *ichthyology*, *limnology*, *alchemy*, and *astrology* using alphabetical order.
- How many comparisons are needed to locate or to add each of these words in the search tree for Exercise 1, starting fresh each time?
 - pear*
 - banana*
 - kumquat*
 - orange*
- How many comparisons are needed to locate or to add each of the words in the search tree for Exercise 2, starting fresh each time?
 - palmistry*
 - etymology*
 - paleontology*
 - glaciology*
- Using alphabetical order, construct a binary search tree for the words in the sentence “*The quick brown fox jumps over the lazy dog.*”
- How many weighings of a balance scale are needed to find a lighter counterfeit coin among four coins? Describe an algorithm to find the lighter coin using this number of weighings.
- How many weighings of a balance scale are needed to find a counterfeit coin among four coins if the counterfeit coin may be either heavier or lighter than the others? Describe an algorithm to find the counterfeit coin using this number of weighings.
- How many weighings of a balance scale are needed to find a counterfeit coin among eight coins if the counterfeit coin is either heavier or lighter than the others? Describe an algorithm to find the counterfeit coin using this number of weighings.
- How many weighings of a balance scale are needed to find a counterfeit coin among 12 coins if the counterfeit coin is lighter than the others? Describe an algorithm to find the lighter coin using this number of weighings.
- One of four coins may be counterfeit. If it is counterfeit, it may be lighter or heavier than the others. How many weighings are needed, using a balance scale, to determine whether there is a counterfeit coin, and if there is, whether it is lighter or heavier than the others? Describe an algorithm to find the counterfeit coin and determine whether it is lighter or heavier using this number of weighings.
- Find the least number of comparisons needed to sort four elements and devise an algorithm that sorts these elements using this number of comparisons.

- *12. Find the least number of comparisons needed to sort five elements and devise an algorithm that sorts these elements using this number of comparisons.

The **tournament sort** is a sorting algorithm that works by building an ordered binary tree. We represent the elements to be sorted by vertices that will become the leaves. We build up the tree one level at a time as we would construct the tree representing the winners of matches in a tournament. Working left to right, we compare pairs of consecutive elements, adding a parent vertex labeled with the larger of the two elements under comparison. We make similar comparisons between labels of vertices at each level until we reach the root of the tree that is labeled with the largest element. The tree constructed by the tournament sort of 22, 8, 14, 17, 3, 9, 27, 11 is illustrated in part (a) of the figure. Once the largest element has been determined, the leaf with this label is relabeled by $-\infty$, which is defined to be less than every element. The labels of all vertices on the path from this vertex up to the root of the tree are recalculated, as shown in part (b) of the figure. This produces the second largest element. This process continues until the entire list has been sorted.



(a)

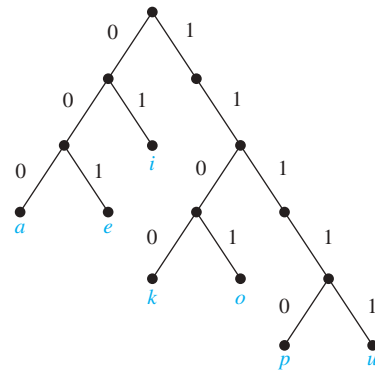


(b)

13. Complete the tournament sort of the list 22, 8, 14, 17, 3, 9, 27, 11. Show the labels of the vertices at each step.
14. Use the tournament sort to sort the list 17, 4, 1, 5, 13, 10, 14, 6.
15. Describe the tournament sort using pseudocode.
16. Assuming that n , the number of elements to be sorted, equals 2^k for some positive integer k , determine the number of comparisons used by the tournament sort to find the largest element of the list using the tournament sort.
17. How many comparisons does the tournament sort use to find the second largest, the third largest, and so on, up to the $(n - 1)$ st largest (or second smallest) element?
18. Show that the tournament sort requires $\Theta(n \log n)$ comparisons to sort a list of n elements. [Hint: By inserting

the appropriate number of dummy elements defined to be smaller than all integers, such as $-\infty$, assume that $n = 2^k$ for some positive integer k .]

19. Which of these codes are prefix codes?
- $a: 11, e: 00, t: 10, s: 01$
 - $a: 0, e: 1, t: 01, s: 001$
 - $a: 101, e: 11, t: 001, s: 011, n: 010$
 - $a: 010, e: 11, t: 011, s: 1011, n: 1001, i: 10101$
20. Construct the binary tree with prefix codes representing these coding schemes.
- $a: 11, e: 0, t: 101, s: 100$
 - $a: 1, e: 01, t: 001, s: 0001, n: 00001$
 - $a: 1010, e: 0, t: 11, s: 1011, n: 1001, i: 10001$
21. What are the codes for a, e, i, k, o, p , and u if the coding scheme is represented by this tree?



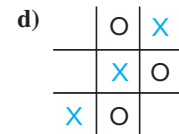
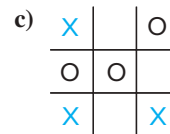
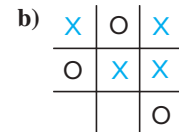
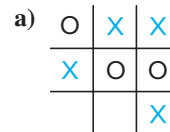
22. Given the coding scheme $a: 001, b: 0001, e: 1, r: 0000, s: 0100, t: 011, x: 01010$, find the word represented by
- 01110100011.
 - 0001110000.
 - 0100101010.
 - 01100101010.
23. Use Huffman coding to encode these symbols with given frequencies: $a: 0.20, b: 0.10, c: 0.15, d: 0.25, e: 0.30$. What is the average number of bits required to encode a character?
24. Use Huffman coding to encode these symbols with given frequencies: $A: 0.10, B: 0.25, C: 0.05, D: 0.15, E: 0.30, F: 0.07, G: 0.08$. What is the average number of bits required to encode a symbol?
25. Construct two different Huffman codes for these symbols and frequencies: $t: 0.2, u: 0.3, v: 0.2, w: 0.3$.
26. a) Use Huffman coding to encode these symbols with frequencies $a: 0.4, b: 0.2, c: 0.2, d: 0.1, e: 0.1$ in two different ways by breaking ties in the algorithm differently. First, among the trees of minimum weight select two trees with the largest number of vertices to combine at each stage of the algorithm. Second, among the trees of minimum weight select two trees with the smallest number of vertices at each stage.
- b) Compute the average number of bits required to encode a symbol with each code and compute the variances of this number of bits for each code. Which tie-breaking procedure produced the smaller variance in the number of bits required to encode a symbol?

27. Construct a Huffman code for the letters of the English alphabet where the frequencies of letters in typical English text are as shown in this table.

Letter	Frequency	Letter	Frequency
A	0.0817	N	0.0662
B	0.0145	O	0.0781
C	0.0248	P	0.0156
D	0.0431	Q	0.0009
E	0.1232	R	0.0572
F	0.0209	S	0.0628
G	0.0182	T	0.0905
H	0.0668	U	0.0304
I	0.0689	V	0.0102
J	0.0010	W	0.0264
K	0.0080	X	0.0015
L	0.0397	Y	0.0211
M	0.0277	Z	0.0005

Suppose that m is a positive integer with $m \geq 2$. An m -ary Huffman code for a set of N symbols can be constructed analogously to the construction of a binary Huffman code. At the initial step, $((N - 1) \bmod (m - 1)) + 1$ trees consisting of a single vertex with least weights are combined into a rooted tree with these vertices as leaves. At each subsequent step, the m trees of least weight are combined into an m -ary tree.

28. Describe the m -ary Huffman coding algorithm in pseudocode.
29. Using the symbols 0, 1, and 2 use ternary ($m = 3$) Huffman coding to encode these letters with the given frequencies: A: 0.25, E: 0.30, N: 0.10, R: 0.05, T: 0.12, Z: 0.18.
30. Consider the three symbols A, B, and C with frequencies A: 0.80, B: 0.19, C: 0.01.
- Construct a Huffman code for these three symbols.
 - Form a new set of nine symbols by grouping together blocks of two symbols, AA, AB, AC, BA, BB, BC, CA, CB, and CC. Construct a Huffman code for these nine symbols, assuming that the occurrences of symbols in the original text are independent.
 - Compare the average number of bits required to encode text using the Huffman code for the three symbols in part (a) and the Huffman code for the nine blocks of two symbols constructed in part (b). Which is more efficient?
31. Given $n + 1$ symbols $x_1, x_2, \dots, x_n, x_{n+1}$ appearing 1, $f_1, f_2, \dots, f_n$ times in a symbol string, respectively, where f_j is the j th Fibonacci number, what is the maximum number of bits used to encode a symbol when all possible tie-breaking selections are considered at each stage of the Huffman coding algorithm?
- * 32. Show that Huffman codes are optimal in the sense that they represent a string of symbols using the fewest bits among all binary prefix codes.
33. Draw a game tree for nim if the starting position consists of two piles with two and three stones, respectively. When drawing the tree represent by the same vertex symmetric positions that result from the same move. Find the value of each vertex of the game tree. Who wins the game if both players follow an optimal strategy?
34. Draw a game tree for nim if the starting position consists of three piles with one, two, and three stones, respectively. When drawing the tree represent by the same vertex symmetric positions that result from the same move. Find the value of each vertex of the game tree. Who wins the game if both players follow an optimal strategy?
35. Suppose that we vary the payoff to the winning player in the game of nim so that the payoff is n dollars when n is the number of legal moves made before a terminal position is reached. Find the payoff to the first player if the initial position consists of
- two piles with one and three stones, respectively.
 - two piles with two and four stones, respectively.
 - three piles with one, two, and three stones, respectively.
36. Suppose that in a variation of the game of nim we allow a player to either remove one or more stones from a pile or merge the stones from two piles into one pile as long as at least one stone remains. Draw the game tree for this variation of nim if the starting position consists of three piles containing two, two, and one stone, respectively. Find the values of each vertex in the game tree and determine the winner if both players follow an optimal strategy.
37. Draw the subtree of the game tree for tic-tac-toe beginning at each of these positions. Determine the value of each of these subtrees.



38. Suppose that the first four moves of a tic-tac-toe game are as shown. Does the first player (whose moves are marked by Xs) have a strategy that will always win?

